



ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA ĐIỆN TỬ – VIỄN THÔNG

BÁO CÁO ĐỒ ÁN

**THIẾT BỊ ĐEO THEO DÕI SỨC KHỎE, PHÁT HIỆN TẾ
NGÃ VÀ ĐỊNH VỊ DÙNG ESP32-S3**

BÁO CÁO CÔNG VIỆC CÁ NHÂN

Người thực hiện	Phạm Hùng Tiến
Mã số sinh viên	24207043
Nhóm	03 thành viên
Phạm vi báo cáo	Tích hợp hệ thống, I2C/OLED, Wi-Fi và giao diện web

Phạm Hùng Tiến – Hồ Sĩ Phú – Quách Gia Thịnh

Mã nguồn đồ án: github.com/PhamHungTien/esp32-s3-health-monitor

Thành phố Hồ Chí Minh, Tháng 8 năm 2026

Nội dung phụ trách

Trong đồ án, em phụ trách cấu hình ESP32-S3 N16R8, giao tiếp I2C, điều khiển OLED SSD1306, Wi-Fi Access Point, dịch vụ HTTP/DNS tự viết, khung dashboard responsive, lịch chạy và kiểm tra hoạt động chung.

1. Mục tiêu công việc

Mục tiêu của phần việc là bảo đảm các cảm biến, GPS, OLED và giao diện web có thể chạy đồng thời trên ESP32-S3. Chương trình cần đọc dữ liệu liên tục, cập nhật hai giao diện đúng chu kỳ, phục vụ yêu cầu HTTP mà không làm tràn FIFO và vẫn giữ được kết nối khi một mô-đun được cắm lại.

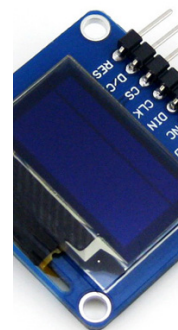
2. Công việc đã thực hiện

2.1. Thiết kế cấu hình phần cứng

Em sử dụng cấu hình bo **ESP32-S3 N16R8**, flash 16 MB, PSRAM 8 MB và chế độ USB CDC khi khởi động. Bus I2C dùng GPIO8 làm SDA và GPIO9 làm SCL. Khi quét bus, chương trình nhận được OLED tại 0x3C, MAX30102 tại 0x57 và IMU tại 0x68. Cụm camera được tách khỏi bus vì các chân 8–9 trùng với tín hiệu camera của bo.



(a) Bo Freenove ESP32-S3-WROOM N16R8.



(b) Màn hình OLED SSD1306 0,96 inch tham chiếu.

Figure 1: Hai linh kiện thuộc phạm vi tích hợp và hiển thị. Nguồn ảnh: Freenove và Waveshare.

2.2. Giao tiếp I2C

Em viết các thao tác đọc/ghi thanh ghi dựa trên Wire có sẵn trong ESP32 Arduino Core, gồm phát START, gửi địa chỉ, ghi thanh ghi, repeated START và đọc nhiều byte. Giá trị trả về được dùng để nhận biết NACK hoặc thiếu dữ liệu. Chương trình thử kết nối lại sau mỗi hai giây nếu một thiết bị I2C bị rút ra.

Đoạn mã dưới đây là thao tác đọc một thanh ghi. Giá trị `false` được truyền lên lớp gọi để mô-đun không tiếp tục sử dụng dữ liệu cũ khi bus báo lỗi.

```

1 // Khai báo hàm đọc một thanh ghi; địa chỉ thiết bị và địa chỉ thanh ghi là đầu vào, còn byte đọc được trả qua biến tham
  // chiếu.
2 bool I2C_ReadRegister(uint8_t address, uint8_t reg, uint8_t &value) {
3
4   // Bắt đầu một giao dịch I2C với thiết bị có địa chỉ đã truyền vào.
5   Wire.beginTransmission(address);
6
7   // Gửi địa chỉ thanh ghi cần đọc vào bộ đệm truyền của bus.
8   Wire.write(reg);
9
10  // Phát repeated START thay vì STOP; nếu thiết bị trả NACK thì kết thúc hàm với trạng thái lỗi.
11  if (Wire.endTransmission(false) != 0) return false;
12
13  // Yêu cầu đúng một byte từ thiết bị; số byte nhận khác một được xem là lỗi truyền.
14  if (Wire.requestFrom(address, (uint8_t)1) != 1) return false;
15
16  // Lấy byte đang chờ trong bộ đệm I2C và ghi vào biến kết quả.
17  value = Wire.read();
18
19  // Xác nhận toàn bộ thao tác đọc đã hoàn tất thành công.
20  return true;
21
22  // Kết thúc thân hàm.
23 }

```

Đoạn mã 1: Đọc thanh ghi I2C và kiểm tra lỗi truyền

2.3. Driver OLED và giao diện

Phần điều khiển SSD1306 sử dụng vùng đệm 1.024 byte và các lệnh ghi trực tiếp qua I2C. Em xây dựng font 5x7 cùng các hàm vẽ điểm, chữ, đường thẳng, khung và biểu tượng tim. Màn hình hai màu được bố trí để 16 hàng màu vàng làm thanh trạng thái, còn 48 hàng màu xanh hiển thị các số đo. Giao diện gồm ba vùng:

- tiêu đề và trạng thái GPS ở đầu màn hình;
- BPM và SpO₂ cỡ lớn ở trung tâm;
- hướng dẫn đo, số bước, trạng thái té ngã, GPS và tọa độ ở cuối màn hình.

Khi có cảnh báo té ngã, màn hình chuyển sang khung cảnh báo riêng, hiện vị trí luân phiên và hướng dẫn hủy cảnh báo. Giao diện không hiển thị số chân đầu dây, nhờ đó thông tin sức khỏe dễ đọc hơn.

Em bổ sung đầu nối piezo hai dây: SPK+ tại GPIO42 và SPK- tại GND. Phần mềm điều khiển PWM LEDC tạo một tiếng ngắn khi khởi động và hai âm xen kẽ khi trạng thái té ngã chuyển sang ALERT. Nếu dùng loa trở kháng thấp thay cho piezo chủ động/thụ động phù hợp, mạch cần thêm transistor khuếch đại.

Framebuffer được truyền ở 400 kHz rồi bus trở lại 100 kHz cho cảm biến. Mỗi giao dịch đều được kiểm tra; nếu OLED không phản hồi, cờ oled0K bị xóa và vòng lặp sẽ thử khởi tạo lại sau hai giây.

2.4. Wi-Fi Access Point và dashboard web

ESP32-S3 tự phát mạng health-monitor với mật khẩu 99999999; địa chỉ truy cập cố định là `http://192.168.4.1`. Chương trình chỉ dùng lớp TCP/UDP mức thấp của ESP32 Arduino Core để điều khiển phần cứng mạng. Nhóm tự phân tích dòng yêu cầu HTTP, tự định tuyến GET/POST, ghép tiêu đề phản hồi, tạo JSON và ghép gói DNS bản ghi A cho captive portal; không dùng WebServer, DNSServer hoặc thư viện web bên ngoài.

Dashboard cập nhật mỗi 0,25 giây và hiển thị BPM, SpO₂, chất lượng PPG, RED/IR thô, số bước, trạng thái té ngã, gia tốc, trạng thái OLED/IMU/MAX30102/buzzer, GPS, vệ tinh, tọa độ, thời gian chạy và số thiết bị đang kết nối. BPM nửa chu kỳ được ghi rõ là sơ bộ cho đến khi có RR đầy đủ. Hai yêu cầu POST cục bộ cho phép đặt lại số bước và hủy cảnh báo té ngã.

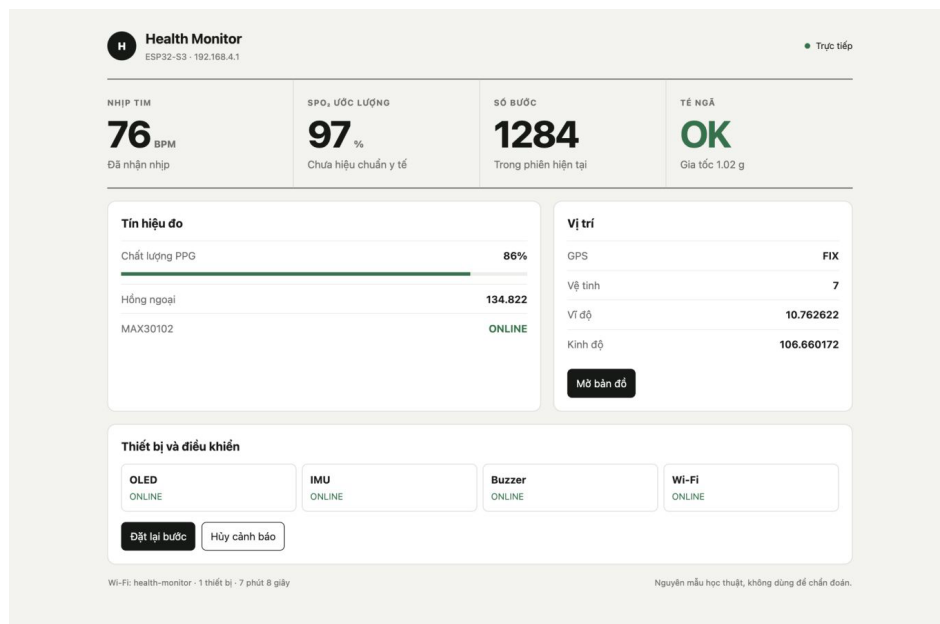


Figure 2: Bản render kiểm tra dashboard tinh gọn với dữ liệu minh họa. Giao diện thực tế nhận số liệu từ API JSON của ESP32-S3 mỗi 0,25 giây.

```

1 // Khai báo bộ định tuyến nhận client và dòng đầu của yêu cầu HTTP.
2 void HTTP_RouteRequest(WiFiClient &client, char *requestLine) {
3     char method[8] = {0};
4     char path[96] = {0};
5
6     // Tách phương thức và đường dẫn ngay từ dòng đầu của yêu cầu HTTP.
7     if (sscanf(requestLine, "%7s %95s", method, path) != 2) {
8         // Yêu cầu không đúng cấu trúc được đưa về trang dashboard an toàn.
9         HTTP_SendDashboard(client);
10        return;
11    }
12
13    // Bỏ chuỗi truy vấn để các phép so sánh chỉ dùng phần đường dẫn chuẩn.
14    char *query = strchr(path, '?');
15    if (query != nullptr) *query = '\0';
16
17    // Định tuyến API trạng thái do nhóm tự viết, không gọi WebServer có sẵn.
18    if (strcmp(method, "GET") == 0 &&

```

```

17     strcmp(path, "/api/status") == 0) {
18     HTTP_SendStatus(client);
19     // Yêu cầu POST thứ nhất đặt bộ đếm bước về 0.
20     } else if (strcmp(method, "POST") == 0 &&
21               strcmp(path, "/api/reset-steps") == 0) {
22         Web_ResetSteps(client);
23     // Yêu cầu POST thứ hai hủy trạng thái cảnh báo té ngã và cập nhật buzzer.
24     } else if (strcmp(method, "POST") == 0 &&
25               strcmp(path, "/api/reset-alert") == 0) {
26         Web_ResetAlert(client);
27     // Mọi đường dẫn lạ và địa chỉ thăm dò captive portal đều nhận dashboard.
28     } else {
29         HTTP_SendDashboard(client);
30     }
31 }

```

Đoạn mã 2: Tự phân tích và định tuyến yêu cầu HTTP

```

1 // Khai báo hàm đẩy framebuffer hiện tại lên OLED và trả về trạng thái truyền.
2 bool OLED_Update() {
3     // Không truy cập bus khi OLED chưa được khởi tạo hoặc đã bị đánh dấu mất kết nối.
4     if (!oledOK) return false;
5     // Nâng tần số I2C lên mức dành cho màn hình để rút ngắn thời gian làm mới.
6     Wire.setClock(I2C_DISPLAY_HZ);
7     // Gửi lệnh chọn phạm vi cột và bắt đầu chuỗi kiểm tra thành công.
8     bool success = OLED_WriteCommand(0x21) && OLED_WriteCommand(0) &&
9     // Đặt cột cuối là 127 rồi chuyển sang lệnh chọn phạm vi trang.
10    OLED_WriteCommand(127) && OLED_WriteCommand(0x22) &&
11    // Chọn các trang từ 0 đến 7, tương ứng toàn bộ vùng 128 x 64 pixel.
12    OLED_WriteCommand(0) && OLED_WriteCommand(7);
13    // Chia framebuffer 1.024 byte thành từng gói 16 byte; vòng lặp dừng ngay khi có lỗi.
14    for (int i = 0; success && i < 1024; i += 16) {
15        // Mở một giao dịch mới tới địa chỉ I2C của SSD1306.
16        Wire.beginTransmission(OLED_ADDR);
17        // Gửi byte điều khiển 0x40 để báo các byte tiếp theo là dữ liệu ảnh.
18        Wire.write(0x40);
19        // Chép 16 byte liên tiếp từ framebuffer vào bộ đệm truyền I2C.
20        for (int j = 0; j < 16; j++) Wire.write(oledBuffer[i + j]);
21        // Phát dữ liệu và lưu kết quả ACK; mã trả về khác 0 làm biến trạng thái thành sai.
22        success = Wire.endTransmission() == 0;
23        // Kết thúc vòng lặp truyền từng gói.
24    }
25    // Hạ bus về tần số dành cho các cảm biến sau khi cập nhật màn hình.
26    Wire.setClock(I2C_SENSOR_HZ);
27    // Trả kết quả để lớp gọi quyết định tiếp tục dùng hay khởi tạo lại OLED.
28    return success;
29    // Kết thúc thân hàm.
30 }

```

Đoạn mã 3: Truyền framebuffer và trả bus về tốc độ cảm biến

2.5. Lịch chạy và tích hợp

Vòng lặp chính dùng mốc `millis()` thay cho trì hoãn dài. GPS được đọc liên tục; FIFO PPG được rút thường xuyên; IMU, OLED và nhật ký nối tiếp chạy ở các chu kỳ riêng. Cách tổ chức này giữ được tốc độ lấy mẫu PPG 100 Hz và tránh mất câu NMEA trong lúc cập nhật màn hình.

```

1  // Phục vụ DNS và HTTP khi Access Point cùng captive portal đã khởi tạo thành công.
2  if (wifiPortalOK) {
3      DNS_Poll();
4      HTTP_Poll();
5  }
6
7  // Đọc toàn bộ byte GPS đang chờ để tránh tràn bộ đệm UART và mất câu NMEA.
8  GPS_Poll();
9
10 // Cập nhật trạng thái phát âm mà không chặn các tác vụ còn lại.
11 Buzzer_Update();
12
13 // Rút tối đa ba nhóm FIFO mỗi vòng để theo kịp luồng PPG nhưng vẫn nhường thời gian cho web.
14 if (maxOK) {
15     for (uint8_t batch = 0; batch < 3; batch++) {
16         int8_t result = MAX30102_ReadFIFO();
17         if (result == FIFO_NO_DATA) break;
18         if (result == FIFO_BUS_ERROR) {
19             maxOK = false;
20             PPG_ResetMetrics();
21             ppg.redRaw = ppg.irRaw = 0;
22             Serial.println("[MAX30102] Mất kết nối I2C");
23             break;
24         }
25     }
26 }
27
28 // Làm sạch MAX30102 và FIFO theo yêu cầu của phiên đo mới hoặc lần bắt BPM lại.
29 if (maxOK && maxReinitRequested) {
30     maxReinitRequested = false;
31     if (MAX30102_Init()) {
32         PPG_ResetMetrics();
33         maxHardwareFreshForSession = true;
34     } else {
35         maxOK = false;
36         maxHardwareFreshForSession = false;
37         Serial.println("[MAX30102] Lỗi khởi tạo lại phiên đo");
38     }
39 }
40
41 // Chỉ xử lý IMU khi khoảng thời gian lấy mẫu quy định đã trôi qua.
42 if (millis() - lastIMUs >= IMU_PERIOD_MS) {
43     // Ghi lại mốc thời gian của lần xử lý IMU hiện tại.
44     lastIMUs = millis();
45
46     // Chỉ đọc gia tốc khi mô-đun IMU đang ở trạng thái hoạt động.
47     if (imuOK) {
48         // Nếu đọc được ba trục gia tốc thì chuyển mẫu sang thuật toán phát hiện té ngã.
49         if (IMU_ReadAccel(ax, ay, az)) Fall_Process(ax, ay, az);
50     }
51     // Đánh dấu và ghi lỗi khi thao tác đọc IMU thất bại để không dùng dữ liệu cũ.

```

```

48     else {
49         imuOK = false;
50         Serial.println("[IMU] Mat ket noi I2C");
51     }
52     // Kết thúc nhánh kiểm tra trạng thái IMU.
53 }
54 // Kết thúc khối tác vụ IMU theo chu kỳ.
55 }
56 // Hủy cờ fix nếu GPS không cập nhật vị trí hợp lệ trong 15 giây.
57 if (gpsFix && millis() - lastGPSFixMs > 15000) gpsFix = false;
58
59 // Sau mỗi hai giây, thử khởi tạo lại từng mô-đun I2C đang mất kết nối.
60 if (millis() - lastProbeMs >= REPROBE_PERIOD_MS) {
61     lastProbeMs = millis();
62     if (!oledOK && I2C_DevicePresent(OLED_ADDR)) {
63         oledOK = OLED_Init();
64         if (oledOK) Serial.println("[OLED] Da ket noi lai");
65     }
66     if (!imuOK && IMU_Init()) {
67         imuOK = true;
68         Serial.println("[IMU] Da ket noi lai");
69     }
70     if (!maxOK && MAX30102_Init()) {
71         PPG_ResetMetrics();
72         maxOK = true;
73         maxHardwareFreshForSession = true;
74         Serial.println("[MAX30102] Da ket noi lai");
75     }
76 }
77 // Kiểm tra đã đến thời điểm làm mới màn hình hay chưa.
78 if (millis() - lastDisplayMs >= DISPLAY_PERIOD_MS) {
79     // Cập nhật mốc thời gian của lần hiển thị hiện tại.
80     lastDisplayMs = millis();
81     // Dựng nội dung trạng thái mới vào framebuffer và truyền lên OLED.
82     OLED_RenderStatus();
83     // Kết thúc khối cập nhật màn hình.
84 }

```

Đoạn mã 4: Các tác vụ có chu kỳ độc lập trong vòng lặp chính

2.6. Khắc phục lỗi biên dịch và nạp

Em cấu hình USBMode=hwcdc, CDCOnBoot=cdc, flash 16 MB và OPI PSRAM cho đúng phiên bản bo. Sau mỗi lần reset, cổng USB mới được kiểm tra lại trước khi nạp. Phiên bản có Wi-Fi cùng HTTP/DNS tự viết sử dụng 969.218 byte flash (30%) và 49.972 byte RAM toàn cục (15%).

3. Kết quả kiểm tra

Kết quả ghi nhận

- ESP32-S3 được nhận đúng revision 0.2, flash 16 MB và PSRAM 8 MB.
- Quét bus trả về đủ ba địa chỉ 0x3C, 0x57, 0x68.
- Nhật ký khởi động báo OLED=OK, IMU=OK, MAX30102=OK và BUZZER=OK. Mục cuối xác nhận kênh LEDC được gán thành công; âm lượng cần kiểm tra trực tiếp theo loại loa.
- GPS được tích hợp vào giao diện; lần chạy cuối báo FIX, 4 vệ tinh và tọa độ hợp lệ.
- Mạng health-monitor khởi tạo ở 192.168.4.1; trang web và API cập nhật đầy đủ trường dữ liệu mà không thay đổi chu kỳ PPG/IMU/OLED.
- Hai thao tác đặt lại số bước và hủy cảnh báo trả JSON thành công; trạng thái mới xuất hiện ở lần cập nhật kế tiếp.
- Firmware nạp thành công qua USB-Serial/JTAG và tự khởi động sau hard reset.

4. Vấn đề và cách xử lý

Vấn đề	Cách xử lý
Không thấy cổng hoặc nạp thất bại	Chọn đúng ESP32-S3, dùng USB native, giảm tốc độ nạp còn 115200 và xác nhận lại cổng sau reset.
Màn hình không hiện dữ liệu	Quét địa chỉ, kiểm tra SDA/SCL, khởi tạo lại SSD1306 và chỉ cập nhật khi truyền I2C thành công.
Giao diện quá nhiều chữ	Ưu tiên hai số sức khỏe, dùng phân vùng và luân phiên tọa độ.
Nhiều mô-đun làm chậm vòng lặp	Chuyển sang lịch không chặn và đọc FIFO/GPS ở mọi vòng lặp.
Trang web làm tăng tải xử lý	Nhúng tài nguyên trong flash, không dùng CDN; API gọn và cập nhật mỗi 0,25 giây; gọi <code>handleClient()</code> theo kiểu không chặn.

5. Kết quả và hạn chế

Phần tích hợp kết hợp OLED và cổng web cục bộ; ba thiết bị I2C, GPS, buzzer cùng dịch vụ HTTP/DNS tự viết được tổ chức trong cùng vòng lặp. Hệ thống hiện thử bằng nguồn USB; cần đo tải Wi-Fi và thời lượng pin, bổ sung xác thực thao tác, hoàn thiện vỏ đeo.